

Penrose Architecture Roadmap

From a Schwarzschild Visualizer to a General Relativity Framework

Status: Planned Architecture

Project: Penrose

Goal: Transform Penrose from a Schwarzschild-specific renderer into a modular computational framework for General Relativity, enabling future research in gravitational lensing, curved spacetime analysis, and Solar Gravitational Lensing (SGL).

Vision

Penrose is evolving beyond a black hole visualization project.

The long-term objective is to become a reusable computational framework capable of simulating arbitrary curved spacetimes while supporting both scientific analysis and real-time visualization.

Rather than designing around a single metric, Penrose will be designed around the mathematical structure of General Relativity itself.

Future applications include:

- Schwarzschild black holes
 - Kerr black holes
 - Weak-field gravitational lensing
 - Solar Gravitational Lens (SGL)
 - Custom spacetime metrics
 - Numerical relativity experiments
 - Dataset generation for machine learning
-

Guiding Principles

The architecture is guided by several core principles.

1. Physics First

The framework should reflect the mathematics of General Relativity rather than the requirements of a renderer.

Every subsystem should correspond to a meaningful physical concept.

Examples include:

- Metrics
 - Coordinate systems
 - Geodesics
 - Observers
 - Emitters
 - Integrators
-

2. Modular Physics

No new spacetime should require modifications to the simulation engine.

Instead, new physics should be introduced by implementing interfaces rather than editing core systems.

3. Separation of Concerns

Physics and rendering solve different problems.

The CPU framework focuses on scientific accuracy and extensibility.

The GPU framework focuses on interactive visualization and rendering performance.

Neither should dictate the other's architecture.

4. Validation Before Optimization

Every new physical capability should first exist in the CPU implementation where it can be validated.

Only after validation should GPU implementations be developed where appropriate.

5. Architecture Evolves, Not Physics

The roadmap does **not** redesign the underlying physics.

The Schwarzschild implementation remains valid.

The architecture simply generalizes the framework so Schwarzschild becomes one supported spacetime among many.

Current State

Current implementation can be summarized as follows.

GPU

- Real-time renderer
- Reduced Schwarzschild metric
- Hardcoded geodesic equations
- Rendering and physics tightly coupled

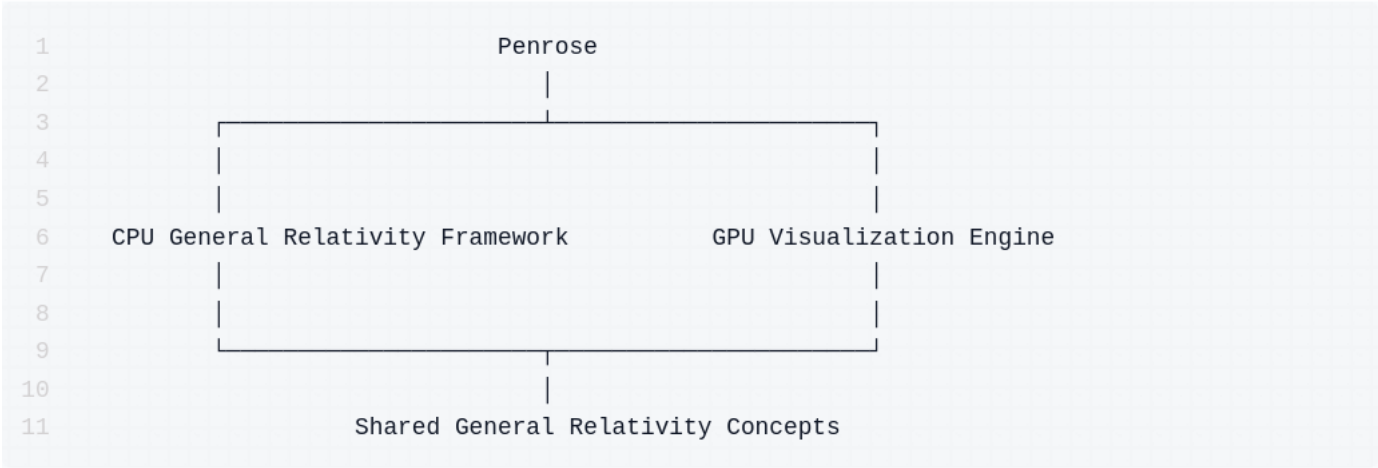
CPU

- Partial refactor
- Beginning modularization
- Independent from GPU
- Intended for future scientific analysis

Overall

- CPU and GPU architectures diverged
- Schwarzschild assumptions exist throughout the codebase
- Framework currently represents one physical model rather than General Relativity as a whole

Target Architecture



The CPU and GPU implementations evolve independently while sharing the same physical definitions.

Shared Physics Layer

The shared layer defines the language of the framework.

This layer should contain no rendering code and no simulation-specific assumptions.

Examples include

```
1  Metric
2  Coordinate System
3  Observer
4  Emitter
5  Geodesic State
6  Physical Constants
7  Units
8  Tensor Utilities
```

Both CPU and GPU consume these definitions independently.

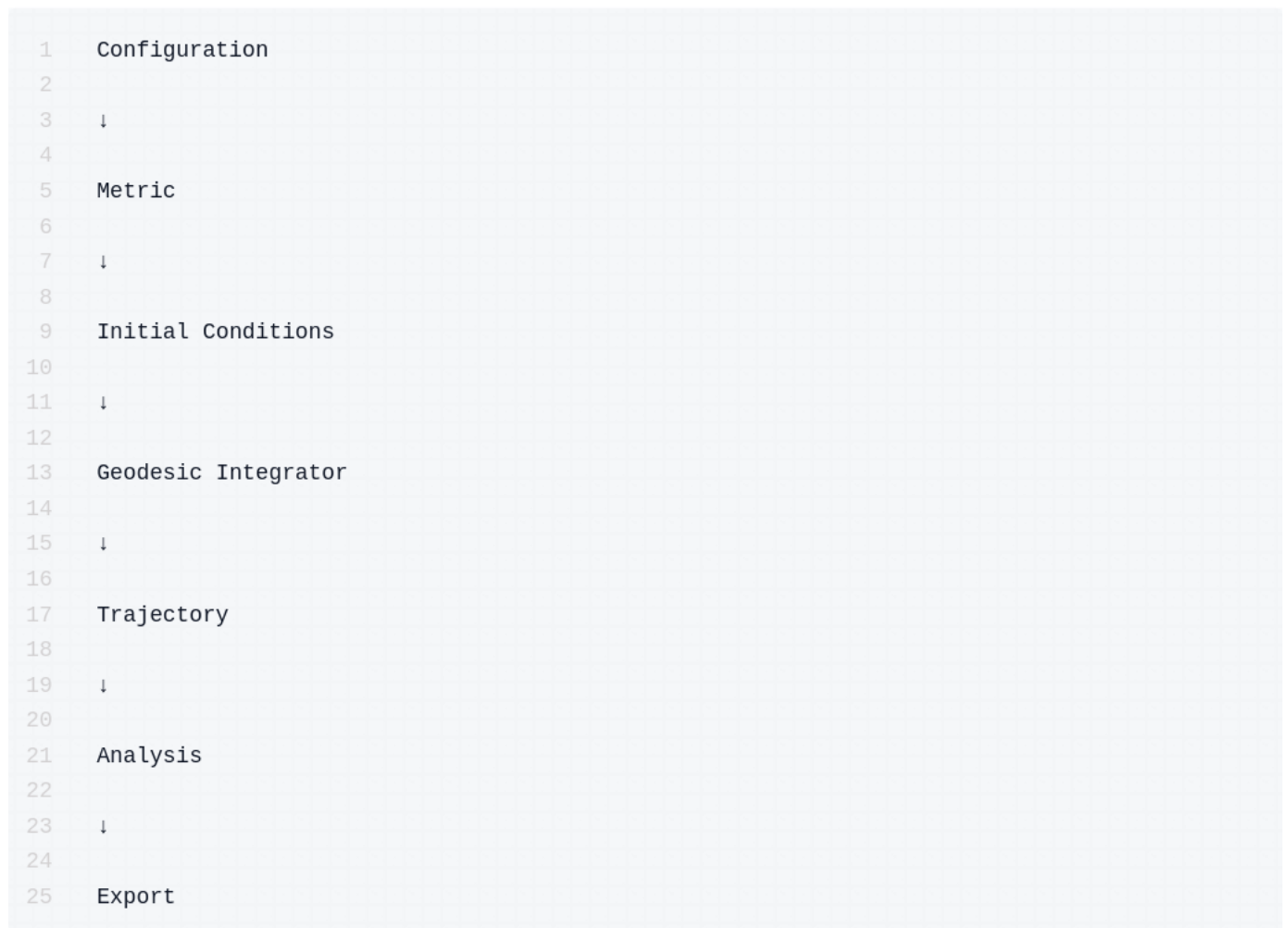
CPU Architecture

The CPU implementation becomes the scientific reference implementation.

Its responsibilities include:

- Accurate geodesic integration
- Validation
- Dataset generation
- Numerical experiments
- Curved spacetime analysis
- Future SGL simulations

The pipeline becomes



Rendering is optional.

GPU Architecture

The GPU implementation focuses exclusively on visualization.

Responsibilities include:

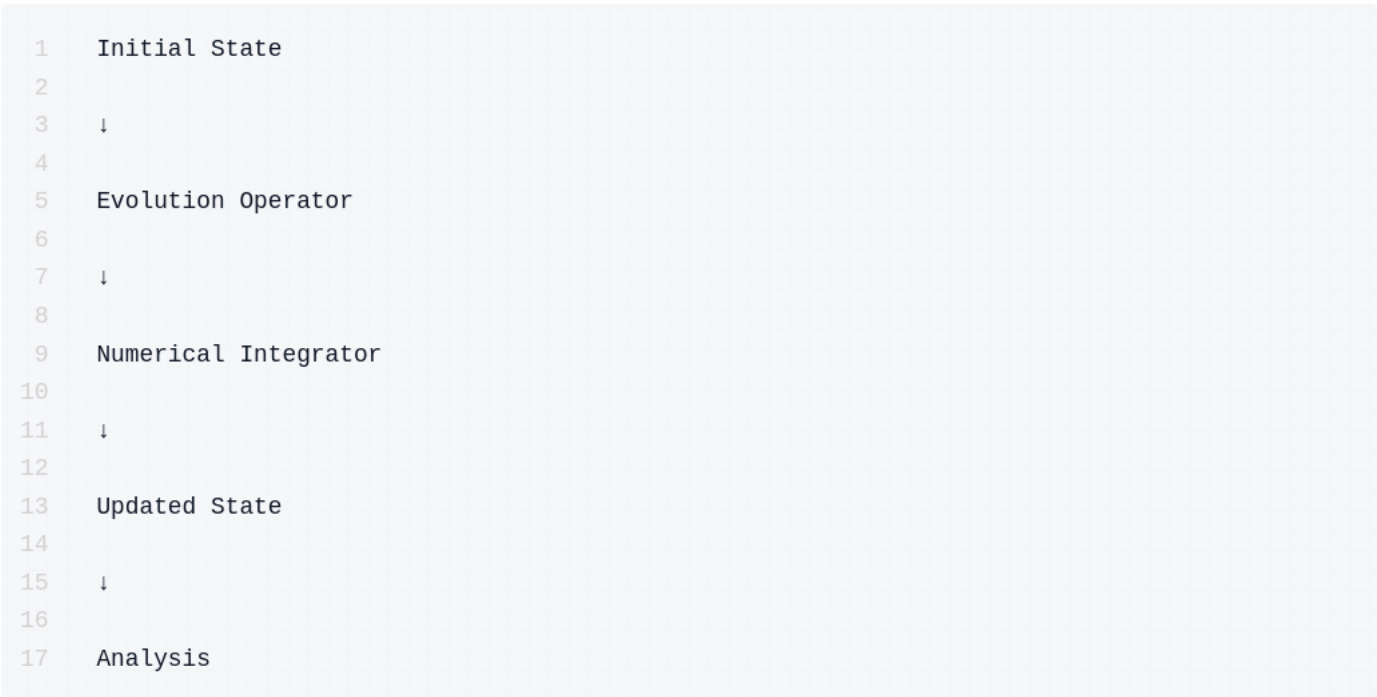
- Interactive rendering
- Camera systems
- Real-time ray tracing
- Visualization tools
- Educational demonstrations

The GPU is not responsible for scientific validation.

Mathematical Architecture

The framework is built around mathematical state evolution.

Every simulation follows the same pattern.



Only the evolution operator changes.

This unifies the architecture with the rest of the scientific software ecosystem.

Core Modules

Metrics

Defines spacetime geometry.

Initially

- Schwarzschild

Future

- Kerr
- Kerr-Newman
- Minkowski
- FLRW
- User-defined metrics

The simulation engine should never require modification when adding a new metric.

Coordinate Systems

Responsible for coordinate transformations.

Examples

- Schwarzschild
- Boyer-Lindquist
- Cartesian
- Isotropic

Metrics should remain independent of coordinate representation whenever possible.

Integrators

Responsible for numerical evolution.

Initial implementations

- RK4

Future

- Adaptive RK
 - Symplectic methods
 - GPU implementations
-

Observers

Defines reference frames.

Examples

- Static observer
 - Freely falling observer
 - Accelerating observer
-

Emitters

Defines light sources.

Examples

- Accretion disk
- Point source
- Background sky
- Star
- Solar surface

Analysis

Transforms trajectories into scientific observables.

Examples

- Deflection angle
- Redshift
- Travel time
- Magnification
- Optical path length
- Detector plane intersections

Rendering becomes one consumer of this data.

Development Philosophy

Architecture uses different paradigms where each is most appropriate.

Object-Oriented Programming

Used to represent physical concepts.

Examples

```
1  Metric
2  Integrator
3  Observer
4  Emitter
5  Simulation
```

These objects encapsulate behavior.

Data-Oriented Design

Used inside numerical kernels.

Examples

- Geodesic state buffers
- Trajectory storage
- Batch integration
- Future SIMD optimizations

Optimization should remain invisible to the higher-level architecture.

Incremental Roadmap

The development of Penrose proceeds along two parallel tracks.

The CPU and GPU implementations are intentionally developed independently during the early stages, allowing each to optimize for its own objectives.

- The CPU evolves into a scientific General Relativity framework.
- The GPU evolves into a modular real-time visualization engine.

Only after both architectures mature do they begin sharing common physics abstractions.

Phase 1 — Independent Foundations

CPU Track

Objective

Establish the CPU implementation as the scientific reference.

Tasks

- Complete CPU refactor
- Remove renderer dependencies
- Standardize simulation pipeline
- Organize project structure
- Centralize physical constants

Deliverable

A standalone CPU Schwarzschild simulation framework focused on correctness and extensibility.

GPU Track

Objective

Establish an independent rendering architecture.

Tasks

- Separate rendering systems from simulation logic
- Clean up rendering pipeline
- Modularize rendering components
- Improve renderer maintainability
- Preserve current visualization capabilities

Deliverable

A standalone GPU rendering engine independent of the CPU implementation.

Phase 2 — Independent Generalization

Both tracks now begin evolving beyond Schwarzschild.

CPU Track

Objective

Generalize the scientific framework.

Tasks

- Introduce Metric abstraction
- Introduce Coordinate System abstraction
- Introduce Observer abstraction
- Introduce Emitter abstraction
- Standardize geodesic state representation

Deliverable

Core simulation engine becomes independent of any specific spacetime.

GPU Track

Objective

Generalize the rendering framework.

Tasks

- Remove renderer assumptions tied specifically to Schwarzschild
- Support modular camera and observer systems
- Prepare renderer for interchangeable spacetime implementations
- Maintain real-time performance

Deliverable

Rendering architecture capable of supporting multiple spacetime implementations without redesign.

Phase 3 — Shared Physics Layer

At this stage, both architectures have matured sufficiently to begin sharing common physical concepts.

Objective

Introduce a common General Relativity layer shared by both CPU and GPU.

Shared modules

- Metric definitions
- Coordinate systems
- Physical constants
- Units
- Tensor utilities
- Geodesic state representation

Each implementation remains free to optimize its own execution model while consuming the same physical definitions.

Deliverable

Independent execution, shared physics.

Phase 4 — Scientific Capability

CPU Track

Objective

Expand scientific functionality.

Tasks

- Trajectory recording
- Deflection angle computation
- Time delay computation
- Redshift computation
- Magnification
- Dataset export
- Batch simulations

Deliverable

Research-grade General Relativity analysis framework.

GPU Track

Objective

Expand visualization capabilities.

Tasks

- Improved visualization tools
- Interactive observer controls
- Multiple rendering modes
- Advanced visualization techniques
- Performance optimization

Deliverable

Feature-rich real-time visualization platform.

Phase 5 — Validation

Objective

Validate the shared physical framework.

Validation targets

- Photon sphere
- Schwarzschild deflection
- Einstein rings
- Shapiro delay
- Published null-geodesic benchmarks

The CPU implementation serves as the scientific reference.

GPU implementations are validated against CPU results where appropriate.

Deliverable

Physically validated framework.

Phase 6 — Framework Expansion

Objective

Demonstrate true modularity.

CPU and GPU should both support additional metrics without modifications to their core architectures.

Possible implementations

- Minkowski
- Kerr
- User-defined metrics

Deliverable

General Relativity framework supporting multiple spacetime geometries.

Phase 7 — Solar Gravitational Lensing

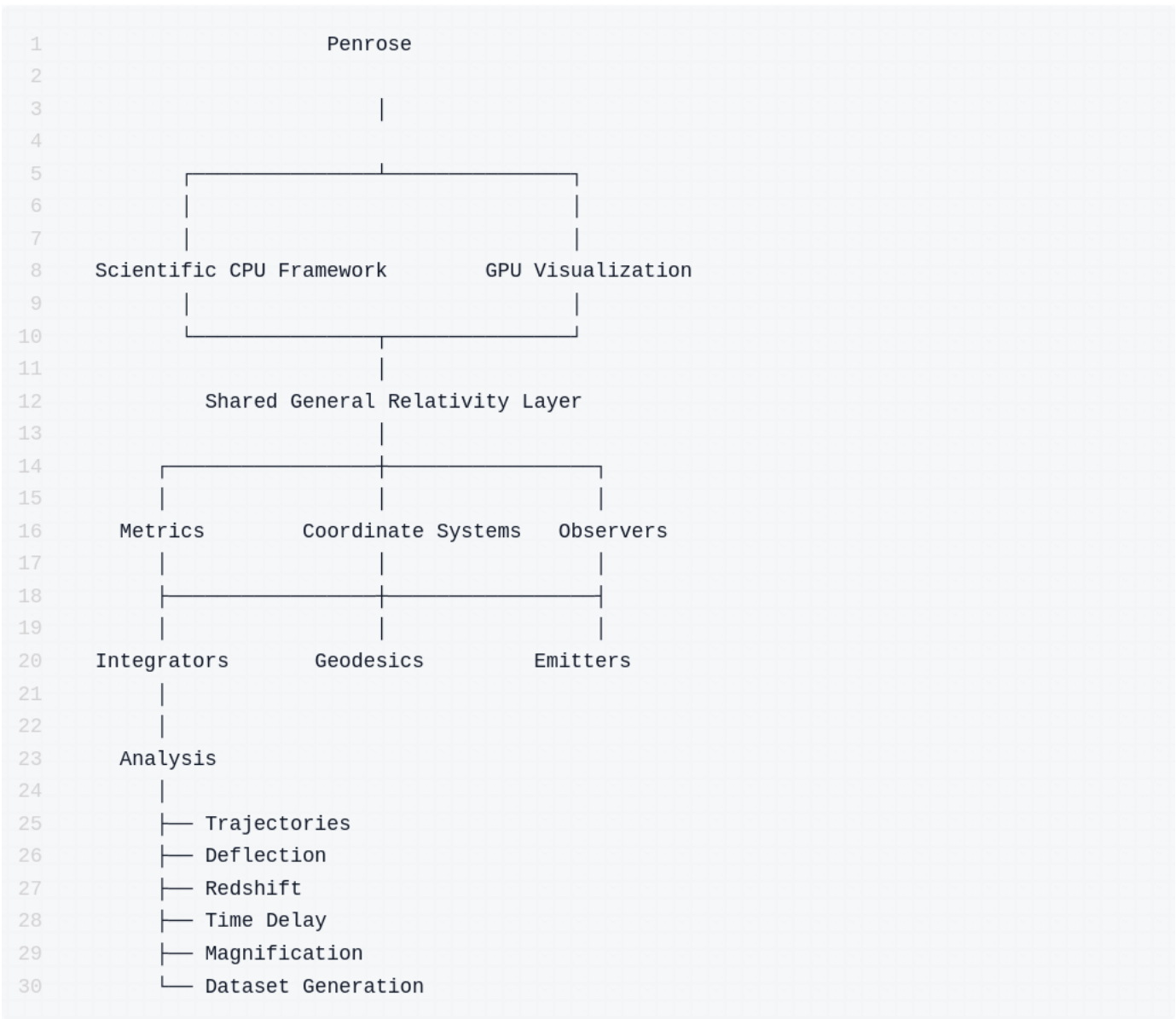
With the framework complete, SGL becomes an application rather than an architectural redesign.

Potential modules include

- Solar metric
- Detector plane
- Telescope model
- Weak-field propagation
- Image reconstruction
- Mission geometry

Both CPU and GPU benefit from the shared General Relativity foundation while remaining independently optimized for their respective objectives.

Long-Term Architecture



The CPU and GPU implementations remain architecturally independent while sharing a common language of General Relativity. This allows each to evolve according to its own requirements without sacrificing long-term consistency across the project.

End Goal

The completed architecture positions Penrose as a modular General Relativity framework rather than a Schwarzschild renderer.

Future research should require implementing new physics modules—not modifying the engine itself.

The framework should support visualization, scientific analysis, numerical experimentation, and future Solar Gravitational Lensing research from the same unified foundation.